In the name of ALLAH

Amirkabir University of Technology
(Tehran Polytechnic)

Deep Neural Networks Course By Dr. Safabakhsh

spring 2025

Assignment (02)

Mohammad Sadegh Eftekhar

403131014

Sadegh.eft@aut.ac.ir

Contents

IMPORTS

1. torch
   For the tensors and the deep learning functionalities

2. torchvision
   For the dataset.

3. torch.nn
   For defining and using neural network layers.

4. torchvision.transforms
   For transforming and preprocessing the image data to tensors.

5. tqdm.auto
   For displaying progress bars during training and evaluation.

6. warning
   For removing any warnings popping while running the code.

7. timeit.default_timer
   For timing the training and evaluation process.

8. pandas
   For making and displaying comparison tables.

9. matplotlib & seaborn
   For visualization of losses and confusion matrix

10. Sklearn.metrics.confusion_matrix
    For generating confusion matrix (as the name suggests)

Before we begin

There are three tasks, that I wanted to perform before getting into the main tasks. The 1st one is removing/ignoring the annoying warnings, which is done using the 'warnings' library.

The 2nd task is setting up a variable which determines the device that we want to perform our calculations on. This is easy to set up, we just check for the 'CUDA' drivers, and if they are installed, we set the 'device' to 'cuda' which means our computation will be done on the 'GPU'. It should be noted that the results reported in this document are derived from the tasks being performed on my personal 'GPU'.

And the 3rd task is to use the 'timeit' library, to define a function, so I can measure the training and evaluation times. (no specific reason, just wanted to see how long it takes). The process is not complicated, whenever we call the 'default_timer' method, it captures the time and stores it in a variable, so all our function does, is just subtract the 'end' time and 'start' time to see how long it took.

Dataset and Transformation to tensors

The code for this section is partially provided by the 'TA', however it was not complete. It is noted in the task instruction, that we have to plot the 'validation' loss, which means the data has to be split into 'train', 'validation' and 'test' sets.

For this reason, I had to make changes to the provided code (not a lot though). What I did was splitting the trainset into 'train' and 'validation' set before creating dataloaders for each of the 3 sets.

The models

In this section I used the 'pytroch' to create the neural networks needed for the tasks. The process of defining the models is pretty generic, noting that the task doesn't want us to make a complex network, this part doesn't really have a lot to explain. I did a little bit of research on the internet, and what I decided to start with, was '5 ReLU' and '5 Linear' between each other for the 'deep neural network' and '2 ReLU' and '2 Linear' for the 'wide neural network'.

Also, to keep the comparison fair as mentioned in the 'homework instruction', we have to make sure the number of learnable weights are fairly close.

However, in the first part of the first question, we are asked to find the best deep and wide models using trial and error, which doesn't make sense to me as keeping the number of weights fair is going to limit the 'finding the best models' task.

The number of input features is going to be the same in both models obviously as each image is 28*28 pixels which when flattened, it would be 784 features.

The number of outputs is fixed as well, because of the number of classes that 'Fasion_MNIST' has, which is 10. The number of features in the hidden layers however, can differ for each model. Knowing the fact

that we want the number of weights to be close to each other, the number of features in the hidden layers have to be greater on the 'wide' model, because of the smaller number of layers.

I asked an 'AI chatbot' for suggestion for the initial value I should use, and I went with '64' for the 'deep neural network' and 80 for the 'wide neural network'. (note that this is the initial value and I will try other values to perform the wanted trial and error)

At first, I didn't really know what I have to do in the 2nd question, after defining the models, I had to modify them because of the 2nd task. The 2nd task, asks us to initialize the weights of our neural networks using 3 different algorithms. What I did to achieve this was to add an extra parameter named 'init_method' which takes-in a string to determine the method we want to use out of the 3.

Then, based on the method we choose, it checks for the linear layers (because the ReLU layers don't have weights) and initializes the weights, and if we don't choose anything it sets it as 'None' and treats the model like there is no weight initialization process. For the 'random' weight initialization I found our what is mostly used is drawing random values from either a 'normal' or a 'uniform' distribution which I used the 'normal' one.

The next step is not special, we define the objects of our model classes and move them to our device of choice (which is 'GPU' in my case).

'Loss' & 'Optimizer'

We now define our loss and optimizers for each model, I decided to keep them the same for that I didn't know what works best for each of them individually. I just looked online to see what is the standard choice for my situation.

For the 'loss', the standard loss function for the multi-class classification tasks is 'Cross entropy loss', and the same goes for the 'Adam' optimizer. The other widely used optimizer is the 'SGD' which is going to be used in the tasks ahead, so I'm not going to use it here.

'Train' & 'Evaluation' Loop

For training, since we're going to have to train multiple models, multiple times, I'm gonna define a function to reuse for all of them. Having used the 'pytorch' a few times before, the training and evaluation part is almost always the same, except some minor differences.

Here we take-in the model, the data loaders (train and validation loaders), the loss function and optimizer, the device and the number of epochs which is set to 8 as default (I found that there is no point in training for more than 8 epochs since the training loss doesn't improve much and the accuracy some times even decreases by a few precents).

To time the training and evaluation processes I used the 'timeit' library (as stated previously), so I decided to use it inside the function instead of using it for individual training codes.

Next, we move to the training loop, we set the model to training mode, send the data to our desired device (in this case 'GPU'), flatten the data, and then perform the well-known training actions of a pytorch training loop.

1. Compute the model prediction.
2. Use the loss function to calculate the loss.
3. Clear the previous gradient
4. Perform backpropagation
5. Update the model weights
6. Calculate the average loss

We then sum add the training loss of the current epoch with the previous losses to keep track of it. We then move on to evaluation steps. we first, set the model to evaluation mode, disable gradient computation for evaluation, repeat the same preprocessing tasks we performed for training loop and enter the evaluation loop.

1. Compute the model prediction (test data)
2. Get the predicted class (using argmax)
3. Store the number for 'total' and 'correct' predictions (for accuracy)
4. Calculate the average loss and accuracy

We then capture the 'end' time, print it and then return the 'train' and 'evaluation' losses for plotting and comparison.

Prediction

The 'prediction' loop is almost the same as the 'evaluation' loop, and since we're always going to use the prediction loop literally after the 'training and evaluation' loop, this has to be a function as well. We use the 'torch.inference_mode' to make sure the gradient computation is off, then make a prediction and turn it into a class label using the argmax. Just like the evaluation loop we're going to keep track of the total number of labels and the number of correct predictions to compute the accuracy.

But we're also going to do something extra, computing a 'confusion matrix' which is done through keep track of all the predictions and all the labels (not just the correct ones). The 'confusion matrix' is computed using the 'sklearn.metrics.confusion_matrix' method.

After that we just return the 'confusion matrix' and the 'accuracy'.

Plotting and Visualization

We're also gonna be doing a lot of plotting and visualization, so this one is better be a function as well. Since we're only going to visualize 3 things ('train loss', 'validation loss', 'confusion matrix'), our visualizer function is also going to take-in those 3 values. We create a '1by2' subplot to fit both plots next to each other. We then use the 1st subplot to visualize the 'train' and 'validation' loss, and the 2nd subplot to show the confusion matrix.

I also tried to summarize the confusion matrix into a 2by2 matrix by adding all the true positives into one true positive and the same with other ones (Tn, Fp, Fn), But it wouldn't work as some values would exceed the total value (and wouldn't make any sense).

Q1.1: find the best 'Wide' and 'Deep' model through trial and error

Like I said in the "The models" section, this question doesn't really make sense to me, because if don't know if it means keep it fair while doing it, meaning the number of weights should be close, or to just ignore being fair and try to find the best parameter values for each of the models. Because if it means the second one, the 'Wide' model can really use more weights in its hidden layer. But then it wouldn't be fair, because the 'Deep' model would have a lot less weights.

However, the process of training and testing and visualizing the results, is not simple thanks to the functions we wrote, we just have to call the functions one-by-one, pass the parameter values, and that's it. We first call the training and evaluation function, we the call the prediction function which takes-in the test dataloader, and next, we take the results of the previous calls and pass them as parameters for the visualizer function to see the results.

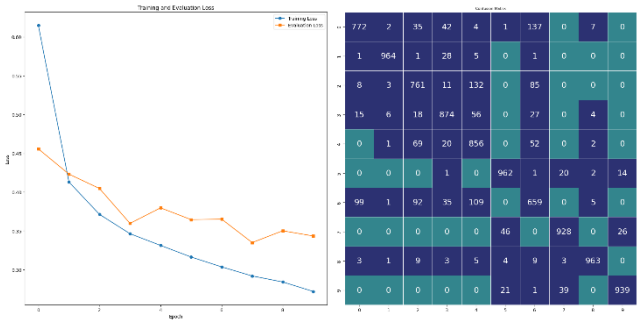
The first set of parameters I used for each model are as follows:

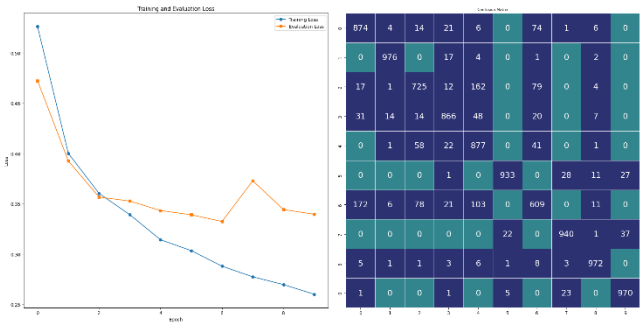
Deep Neural Network:

hidden features: 64
epochs: 10
total number of parameters: 63,530
evaluation accuracy: 87.93%
Test accuracy: 86.92%
run time on GPU: 9.202 minutes

Wide Neural Network:

hidden features: 80
epochs: 10
total number of parameters: 63,370
evaluation accuracy: 88.17%
Test accuracy: 87.85%
tun time on GPU: 9.730 minutes



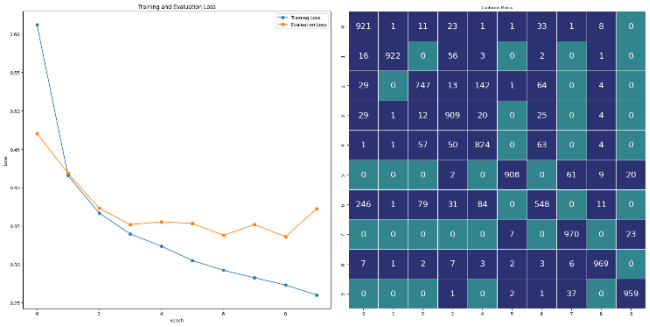


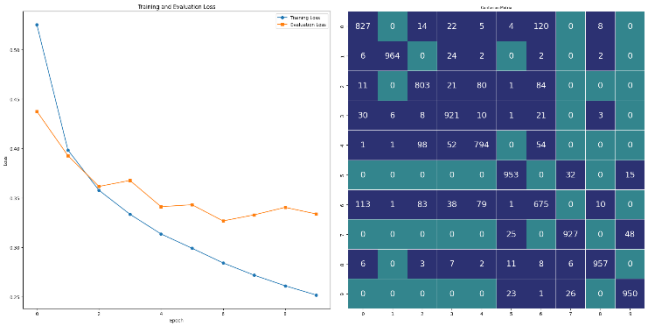
Deep Neural Network:

hidden features: 76
epochs: 10
total number of parameters: 78,206
evaluation accuracy: 88.05%
Test accuracy: 87.08%
run time on GPU: 9.295 minutes

Wide Neural Network:

hidden features: 100
epochs: 10
total number of parameters: 79,510
evaluation accuracy: 88.26%
Test accuracy: 87.21%
tun time on GPU: 8.484 minutes





The results did improve a really really small amount, but I don't know if it did or didn't worth the increase in computation time, it took between 1 to 2 minutes extra to run each model this time, which can add up when running a lot of them.

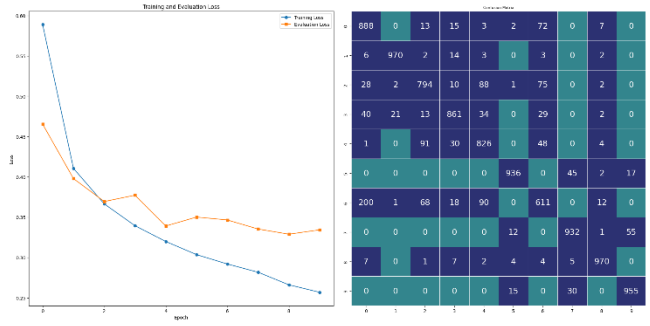
Lets increase them only one more time:

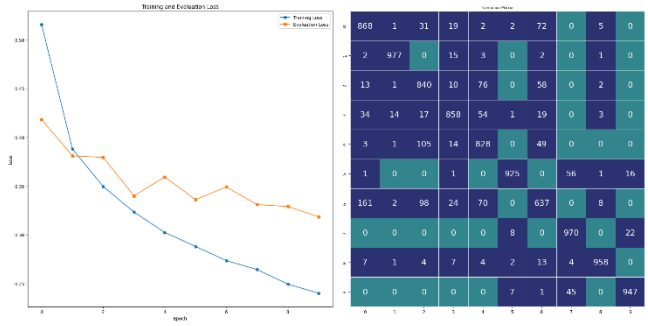
Deep Neural Network:

Hidden features: 104
Epochs: 10
Total number of parameters: 112,618
Evaluation accuracy: 88.26%
Test accuracy: 87.73%
run time on GPU: 8.896 minutes

Wide Neural Network:

Hidden features: 136
Epochs: 10
Total number of parameters: 112,970
Evaluation accuracy: 88.67%
Test accuracy: 88.08%
run time on GPU: 8.338 minutes





Q1.2: compare the performance of the models, which one is better fitted and why ?

I was surprised seeing the results are quite close, because I wouldn't have thought a model with only one layer could have this much potential. Searching on the internet for the strength and weaknesses of 'deep' and 'wide' neural networks, I found out that:

- deep networks have more capacity to learn complex patterns due to their multiple layers. However, they are also more prone to overfitting, especially if the dataset is not large enough.
- Wide networks, with fewer layers but more neurons per layer, can capture broader patterns but might struggle with more intricate details compared to deep networks.

Which does explain why the 'wide' neural network is performing better than I expected noting that the 'FasionMNIST', is a pretty simple dataset, with not a lot of features, which doesn't require a lot of layers to learn.

They're close in almost all the categories, it just seems like as the number of parameters (weights) increases the 'wide' model outperforms the 'deep' model even more in all the categories (time, accuracy, loss), but it's not a huge leap and they perform quite the same.

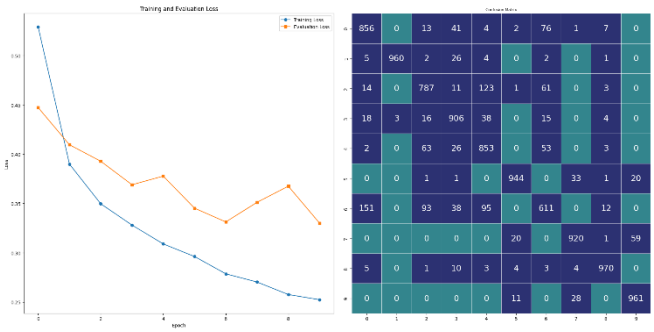
Q2.1: Initialize the weights using the 'He', the 'Xavier' a and randomly, then, Compare and analyze the results

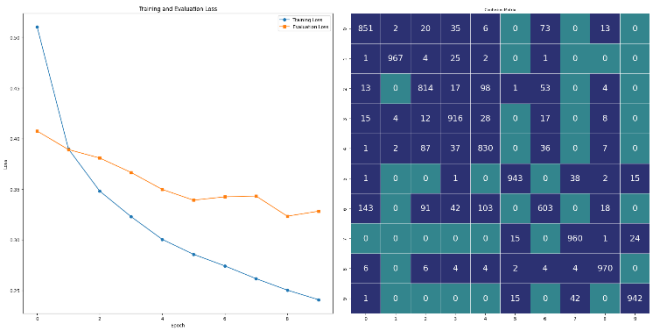
As I said in the "The models" section, I added the ability to pass one of the mentioned algorithms as parameter to the model, so it's used in the weight initialization. So the process of this task is done already and we just need to call each model 3 times, with 3 different weight initialization methods, and compare the results.

The Results different 'weight initialization algorithms' on:

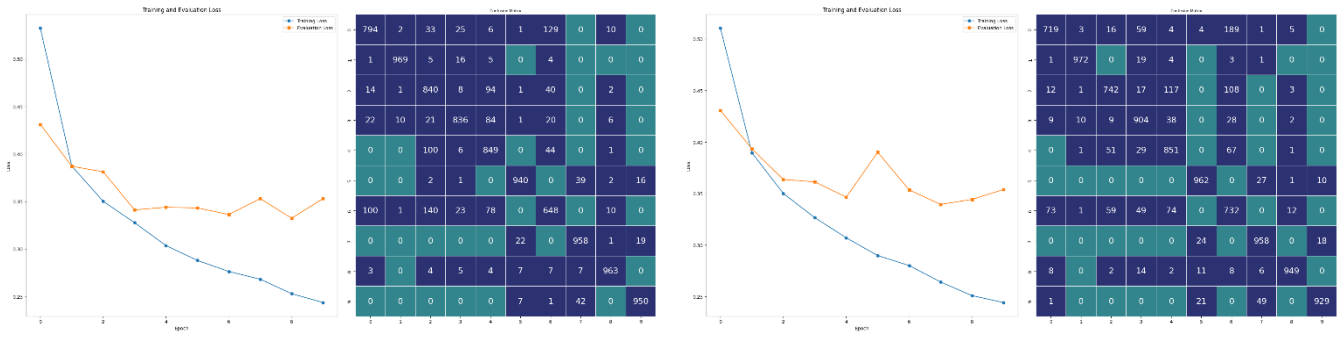
Deep Neural Network **Wide Neural Network**

He

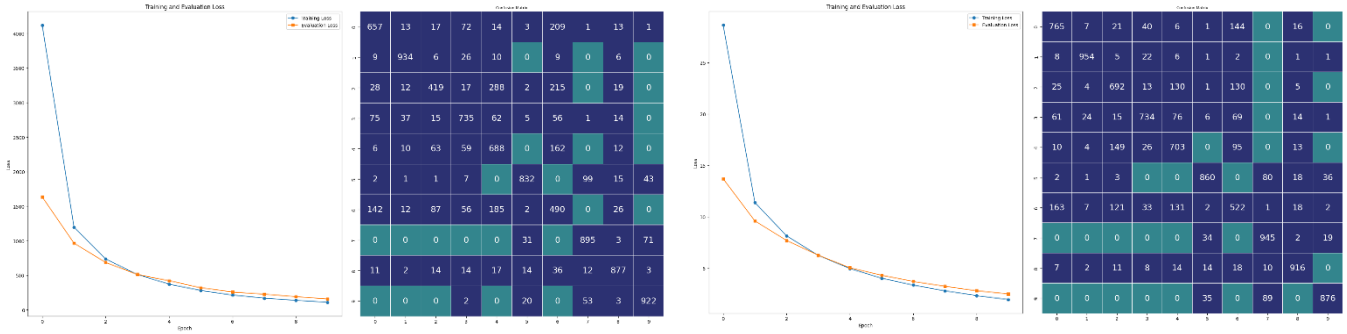




Xavier



Random



Comparison

The accuracy table:

	DeepNN	WideNN
He	87.68	87.96
Xavier	87.47	87.18
Random	74.49	79.67

As can be seen the ‘Wide’ model outperformed the ‘Deep’ model in 2 of the weight initialization methods ‘He’ and ‘Random’ but failed in ‘Xavier’. The performance leap in ‘Random’ is huge, with about 5% better performance. However, the performance of the ‘Random’ method is not as good as the other 2.

The best performing method here is the ‘He’ algorithm, which as mentioned the ‘Wide’ model outperformed the ‘Deep’ model in.

Q2.2: state the strengths and weaknesses of these algorithms based on the mathematical equation and the results of the experiments

Algorithms / Features	He	Xavier	Random (Normal dist)
Accuracy	Deep: 87.68% Wide: 87.66%	Deep: 87.47% Wide: 87.18%	Deep: 74.49% Wide: 79.67%
Designed for	ReLU	Sigmoid / Tanh	None (Uncontrolled)
Key strengths	Best for 'ReLU' networks	Best for 'Sigmoid / Tanh' Variance Stability	Simple Quick
Key weaknesses	Minor overfitting Not good for Tanh networks	Not Ideal for 'ReLU'	Unstable Gradient Uneven weight distribution

Q3.1: Experiment with different learning rates (0.001, 0.01, 0.1, 1.0)

For this I made a new function which only takes in one parameter, with the purpose of determining if which of the models we want to experiment with, if we call the function and pass 'True', it uses the 'deep neural network' for the learning rate experiment, and otherwise it uses the 'wide neural network'.

Aside from the model we choose the rest of the process is the same, the function defines a list of different learning rates and a loss function, then in a loop, if defines the model (based on the binary parameter), defines an optimizer, then uses the 'train and evaluation', 'prediction' and 'visualization' functions.

The model then, returns the accuracy (for comparison).

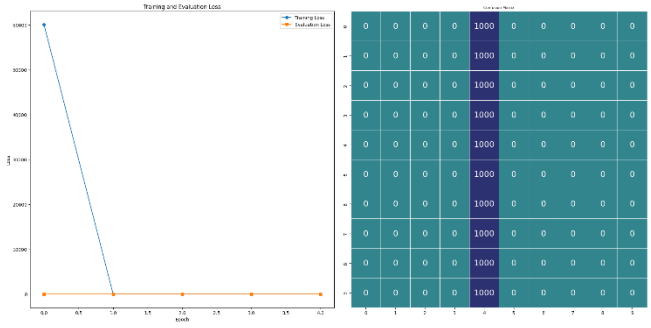
After defining the function, we now can call it twice, once for the 'deep' and another time for the 'wide' model. Then, we can use the 'pandas' library to show the results in a table.

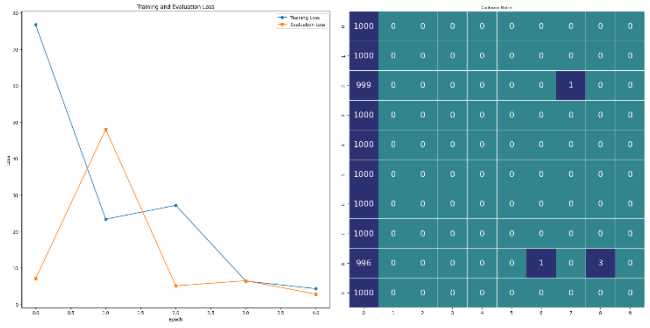
The Results different ‘Learning Rates’ on:

Deep Neural Network

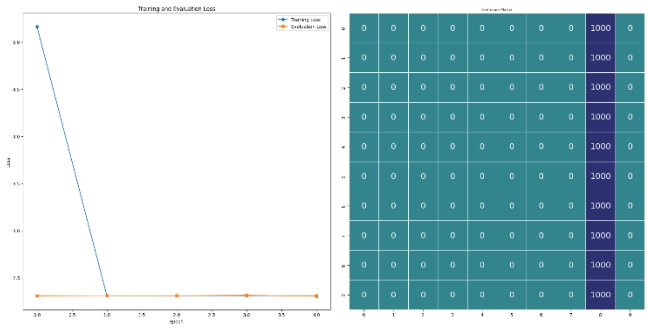
Wide Neural Network

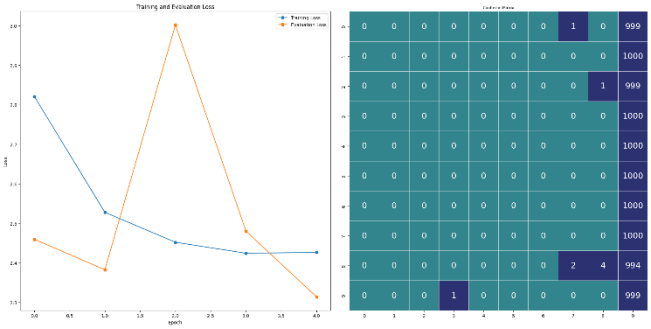
1.0



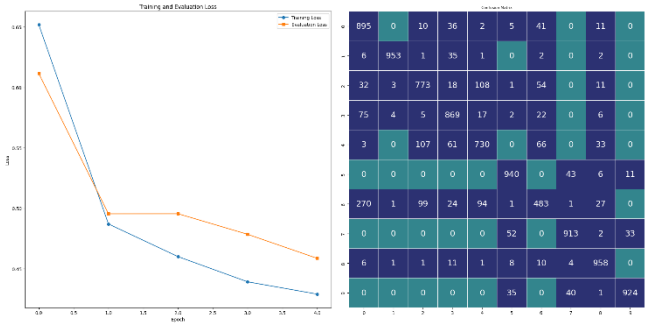


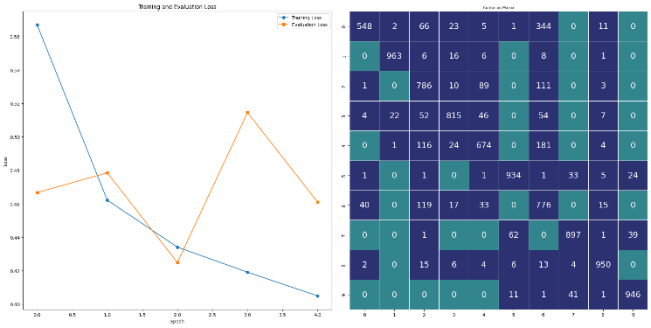
0.1



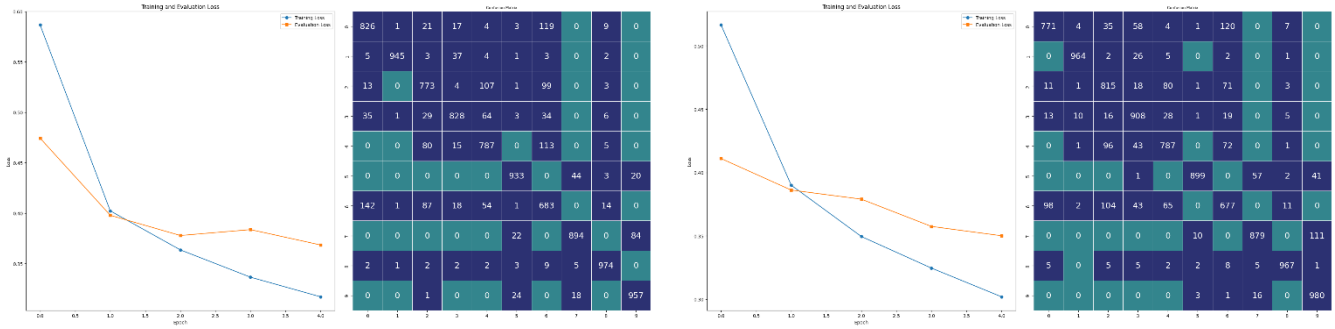


0.01





0.001



comparison

	Learning Rate	DeepNN Accuracy	WideNN Accuracy
0	1.000	10.00	10.03
1	0.100	10.00	10.03
2	0.010	84.38	82.89
3	0.001	86.00	86.47

Not a lot to talk about, the accuracies are showing an obvious result, the '0.001' outperforms all of the other 'learning rates' in both the 'Deep neural network' and the 'Wide neural network'.

Q3.2: Change the batch sizes (32, 64, 128, 256) and observe how the accuracy of the models changes.

As the concept is really close to the previous question (change something and see what happens), the method of doing it is the same. We define a function which takes-in a binary value to determine which model we want to experiment on, keeps a list of the batch sizes we want to try, defines a loss function, and then enters a loop to repeat the process for each batch size in the list.

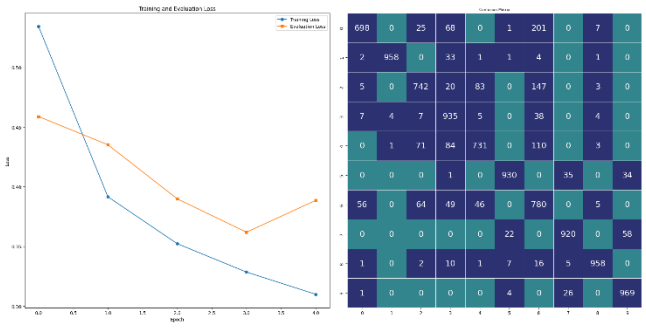
The only thing extra here compared to the last function, is we have to also define the dataloaders again, because we want to change the batch size value, the rest is pretty much the same, define model, define optimizer, train, validate, predict and visualize.

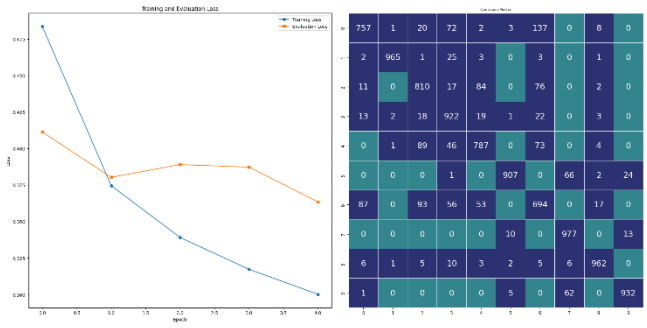
The Results different ‘Batch Sizes’ on:

Deep Nueral Networks

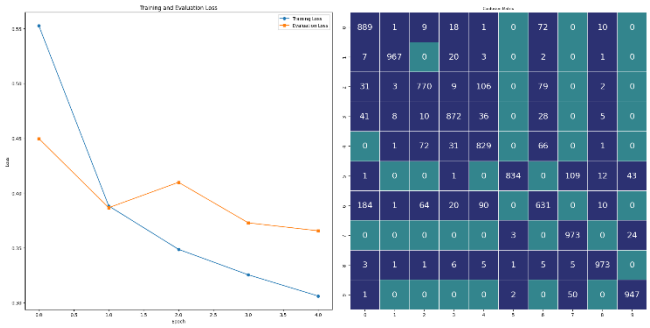
Wide Neural Networks

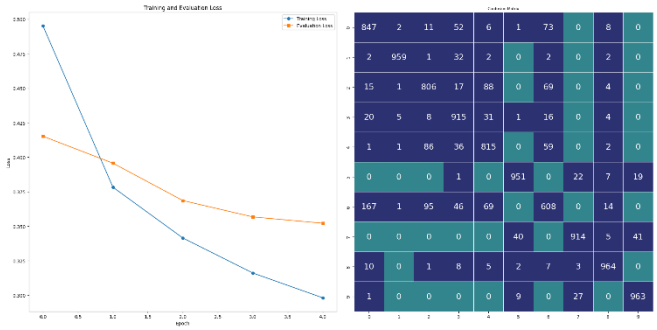
32



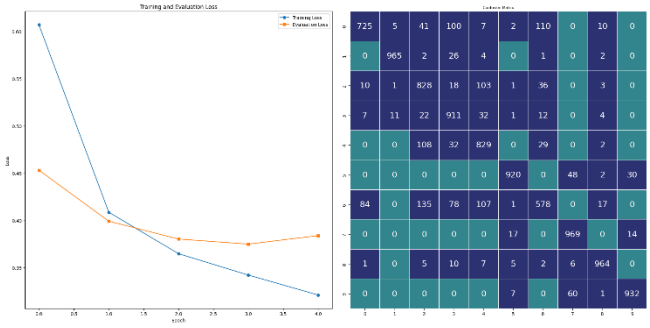


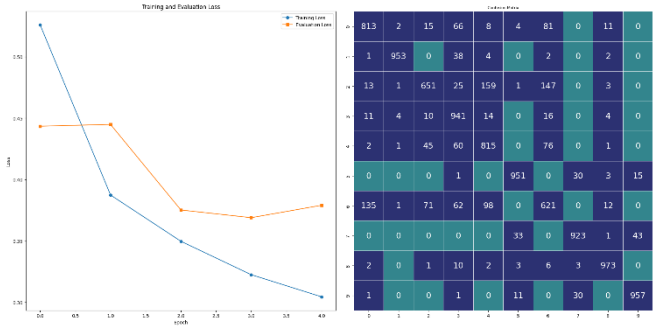
64



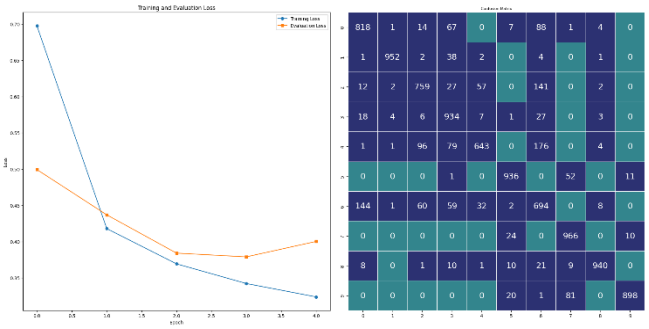
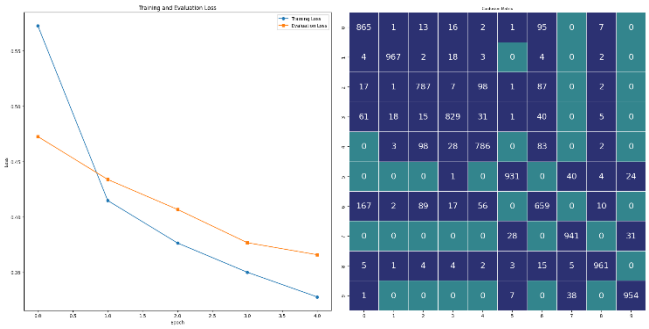


128





256

comparison

	Batch size	DeepNN Accuracy	WideNN Accuracy
0	32	86.21	87.13
1	64	86.85	87.42
2	128	86.21	85.98
3	256	85.40	86.80

As can be seen in the table above, the ‘batch size’ of ‘64’ is the optimal choice, outperforming the other 3 batch sizes both on the ‘Deep neural network’ and the ‘Wide neural network’.

Q3.3: try ‘Adam’ and ‘SGD’ optimizers and compare the results

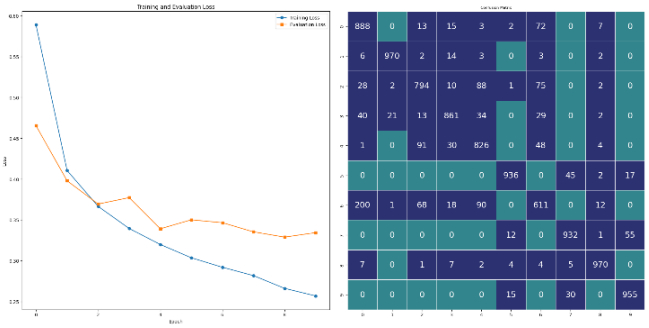
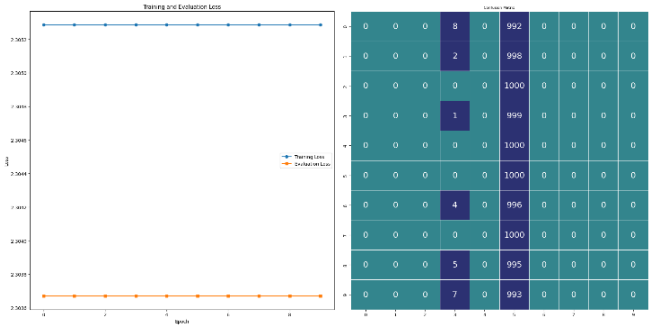
Since I did all of the previous steps using the ‘Adam’ optimizer, we only need to repeat some of them with the ‘SGD’ optimizer and do a comparison. Those steps are defining the loss functions and optimizers, training and validating, predicting on the test set, visualize and then make a table for the accuracies for easier comparison.

The Results different ‘optimizers’ on:

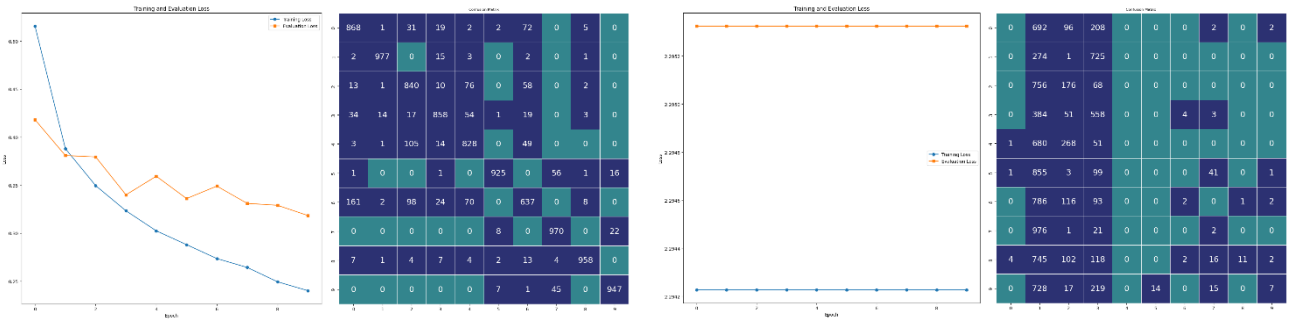
Deep Nueral Networks

Wide Neural Networks

Adam

SGD



Comparison

	Optimizer	DeepNN	WideNN
0	SGD	10.01	10.30
1	Adam	87.43	88.08

The winner is pretty clear, the ‘Adam’ optimizer outperformed the ‘SGD’ optimizer by a huge leap, the ‘SGD’ performed so bad, that if we flip the labels of it, we can achieve around 90% accuracy. I was expecting a better performance from ‘Adam’ knowing it’s a better choice for what we’re aiming to here, but wouldn’t have thought such a big difference.